

学号： 2106410113

大数据技术基础

作业

学 生 姓 名： 张艳芳 专 业 班 级： 计算机 2101

第一章

1. 代码+注释

创建一个空列表来存储姓名

```
name = []
```

提示用户输入姓名，用逗号分隔

```
n = input("输入姓名（用,隔开）：")
```

用 split() 方法将输入的字符串按逗号分割成单个姓名并存储在 'name' 列表中

```
name = n.split(',')
```

提示用户输入要搜索的姓名

```
n = input("检索姓名：")
```

使用条件语句检查输入的姓名是否存在于 'name' 列表中

```
if n in name:
```

```
    print('姓名在列表中')
```

```
else:
```

```
    print('姓名不在列表中')
```

2. 程序说明

该程序的功能是接收用户输入的姓名，并将其存储在一个列表中。然后，用户可以输入一个要搜索的姓名，程序将检查该姓名是否存在于列表中，并相应地输出结果。

3. 截图

```
In [53]: runfile('D:/大数据python/1.2.py', wdir='D:/大数据python')
```

输入姓名（用,隔开）：张艳芳,崔韩率,尹净汉,金珉奎

检索姓名：张艳芳

姓名在列表中

```
In [54]: runfile('D:/大数据python/1.2.py', wdir='D:/大数据python')
```

输入姓名（用,隔开）：张艳芳,崔韩率,尹净汉,金珉奎

检索姓名：李硕珉

姓名不在列表中

第二章

1. 代码+注释

导入 NumPy 库

```
import numpy as np
```

创建超市和水果的数组

```
supermarket = np.array(['大润发', '沃尔玛', '好德', '农工商'])
```

```
fruit = np.array(['苹果', '梨', '香蕉', '橘子', '芒果'])
```

使用 NumPy 的随机整数生成函数 randint() 创建一个形状为 (4, 5) 的二维数组 price，表示不同超市不同水果的价格。

```
price = np.random.randint(4, 11, size=(4, 5))
```

打印不同超市水果价格

```

print('不同超市水果价格:')
print(price)
# 通过索引和布尔运算, 将大润发苹果和好德香蕉价格增加 1 元
price1 = price[(supermarket == '大润发') | (supermarket == '好德'),
               (fruit == '苹果') | (fruit == '香蕉')] + 1
print('将大润发苹果和好德香蕉价格增加 1 元:', price1)
# 通过索引和布尔运算, 将农工商所有水果价格减少 2 元
price2 = price[supermarket == '农工商'] - 2
print('将农工商所有水果价格减少 2 元:', price2)
# 计算苹果和芒果的销售均价
price3 = price[:, (fruit == '苹果') | (fruit == '芒果')].mean()
print('苹果和芒果的销售均价:', price3)
# 找出橘子价格最贵的超市
name = price[:, fruit == '橘子'].argmax()
print('橘子价格最贵的超市:', supermarket[name])

```

2. 程序说明

该程序利用 NumPy 数组的索引、布尔运算和统计计算功能, 对不同超市水果的价格进行操作和计算, 并输出相应的结果。

- 1) `mean()`: 用于计算数组的均值, 用来计算苹果和芒果的销售均价。
- 2) `argmax()`: 用于找到数组中最大元素的索引, 用来找到橘子价格最贵的超市的名称。
- 3) `np.array()`: 用于创建 NumPy 数组, 将超市名称和水果名称保存为 NumPy 数组。
- 4) `np.random.randint()`: 用于生成指定范围内的随机整数, 用来模拟不同超市不同水果的价格。

3. 截图

```

In [55]: runfile('D:/大数据python/2.1.py', wdir='D:/大数据python')
不同超市水果价格:
[[ 9  5 10  4 10]
 [ 4  6  9 10  9]
 [ 6  6  8  8  8]
 [ 4  9  7  9  9]]
将大润发苹果和好德香蕉价格增加1元: [10  9]
将农工商所有水果价格减少2元: [[2 7 5 7 7]]
苹果和芒果的销售均价: 7.375
橘子价格最贵的超市: 沃尔玛

```

第三章

1. 代码+注释

```

import pandas as pd
# 读取 CSV 文件并创建 DataFrame 对象
bank = pd.read_csv('bankpep.csv')
# 计算并打印储户总数
print('储户总数:', bank['id'].count())
# 按区域分组并计算每个区域的储户数

```

```

grouped = bank.groupby(['region'])
print('分区储户数：')
print(grouped['id'].count())
# 按性别分组并计算不同性别储户收入的均值和方差
grouped1 = bank.groupby(['sex'])
print('不同性别储户收入均值：')
print(grouped1['income'].mean())
print('不同性别储户收入方差：')
print(grouped1['income'].var())
# 按性别和区域分组并计算接受新业务的储户数
grouped2 = bank.groupby(['sex', 'region'])
print('接受新业务的储户数(按性别、区域统计)：')
print(grouped2['pep'].count())
# 将存款账户和接受新业务的数据进行映射转换，将'YES'映射为 1，'NO'映射
为 0
bank['save_act'] = bank['save_act'].map({'YES': 1, 'NO': 0})
print('存款账户数据转换结果：')
print(bank['save_act'])
bank['pep'] = bank['pep'].map({'YES': 1, 'NO': 0})
print('接受新业务数据转换结果：')
print(bank['pep'])
# 计算收入、存款账户和接受新业务之间的相关性
relationship = bank[['income', 'save_act', 'pep']].corr().round(2)
print('收入、存款账户、接受新业务的关系：')
print(relationship)

```

2. 程序说明

- 1) 统计储户总数：通过使用 DataFrame 的 count() 函数统计 bank 对象中'id'列的数据数量，即储户的总数，并将结果打印输出。
- 2) 按区域分组并计算每个区域的储户数：使用 DataFrame 的 groupby() 函数按照'region'列进行分组，然后使用 count() 函数统计每个分组中'id'列的数据数量，即每个区域的储户数，并将结果打印输出。
- 3) 按性别分组并计算不同性别储户收入的均值和方差：使用 DataFrame 的 groupby() 函数按照'sex'列进行分组，然后使用 mean() 函数和 var() 函数分别计算每个分组中'income'列的均值和方差，即不同性别储户收入的均值和方差，并将结果打印输出。
- 4) 按性别和区域分组并计算接受新业务的储户数：使用 DataFrame 的 groupby() 函数按照'sex'列和'region'列进行分组，然后使用 count() 函数统计每个分组中'pep'列的数据数量，即接受新业务的储户数，并将结果打印输出。
- 5) 将存款账户和接受新业务的数据进行映射转换：使用 DataFrame 的 map() 函数将'save_act'列中的'YES'映射为 1，'NO'映射为 0，将'pep'列中的'YES'映射为 1，'NO'映射为 0。这样可以将存款账户和接受新业务的数据转换为数值型。
- 6) 计算收入、存款账户和接受新业务之间的相关性：使用 DataFrame 的 corr() 函数计算'bank'对象中'income'、'save_act'和'pep'三列之间的相关系数，

并使用 round(2) 函数将结果保留两位小数。最后,将相关系数矩阵打印输出。

3. 截图

```
In [56]: runfile('D:/大数据python/3.2.py', wdir='D:/大数据python')
```

储户总数: 600

分区储户数:

region

INNER_CITY 269

RURAL 96

SUBURBAN 62

TOWN 173

Name: id, dtype: int64

不同性别储户收入均值:

sex

FEMALE 27831.368233

MALE 27216.694200

Name: income, dtype: float64

不同性别储户收入方差:

sex

FEMALE 1.698137e+08

MALE 1.633458e+08

Name: income, dtype: float64

接受新业务的储户数(按性别、区域统计):

sex	region	
FEMALE	INNER_CITY	131
	RURAL	47
	SUBURBAN	30
	TOWN	92
MALE	INNER_CITY	138
	RURAL	49
	SUBURBAN	32
	TOWN	81

Name: pep, dtype: int64

存款账户数据转换结果:

0	0
1	0
2	1
3	0
4	1
..	
595	1
596	1
597	1

```

598     1
599     0
Name: save_act, Length: 600, dtype: int64
接受新业务数据转换结果:

```

```

0     1
1     0
2     0
3     0
4     0

```

```

..
595    0
596    0
597    1
598    0
599    1

```

```

Name: pep, Length: 600, dtype: int64
收入、存款账户、接受新业务的关系:

```

	income	save_act	pep
income	1.00	0.27	0.22
save_act	0.27	1.00	-0.07
pep	0.22	-0.07	1.00

第四章

1. 代码+注释

```

import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

```

```

# 从"bankpep.csv"文件中读取用户信息
bank = pd.read_csv('bankpep.csv')

```

```

# 绘制客户年龄分布的直方图和密度图
bank['age'].plot(kind='hist', bins=10, density=True, title='Customer
Age Distribution') # 绘制直方图
bank['age'].plot(kind='kde', title='Customer Age Distribution',
style='k--') # 绘制密度图
plt.xlabel('Age')
plt.ylabel('Density')
plt.show()

```

```

# 绘制客户年龄和收入的散点图
bank[['age', 'income']].plot(kind='scatter', x='age', y='income',
title='Customer Income', marker='*', grid=True, xlim=[0, 80], ylim=[0,
70000], label='(age, income)')
plt.xlabel('Age')
plt.ylabel('Income')
plt.show()

```

```

# 绘制年龄、收入和孩子数的散点矩阵图

```

```

data = bank[['age', 'income', 'children']]
pd.plotting.scatter_matrix(data, diagonal='hist', c='#47619E')
plt.show()

# 按区域分组，计算收入的均值和标准差，并绘制柱状图
mean = bank.groupby(['region']).agg({'income': np.mean}) # 按区域分
组计算收入均值
std = bank.groupby(['region']).agg({'income': np.std}) # 按区域分组
计算收入标准差
mean.plot(kind='bar', yerr=std, rot=0, title='Customer Income',
legend=False, color='#47619E') # 绘制柱状图
plt.xlabel('Region')
plt.show()

# 绘制性别、有车客户和孩子数的饼图
data1 = bank.groupby(['sex'])['sex'].count() # 按性别统计客户数量
data2 = bank[bank['car'] == 'YES'].groupby(['sex'])['sex'].count() #
按性别统计有车客户数量
data3 = bank.groupby(['children'])['children'].count() # 按孩子数统
计客户数量

fig = plt.figure(figsize=(7, 6))
fig.add_subplot(2, 2, 1)
data1.plot(kind='pie', title='Customer Sex', startangle=60,
autopct='%1.1f%%') # 绘制性别的饼图
fig.add_subplot(2, 2, 2)
data2.plot(kind='pie', title='Customer Car Sex', startangle=60,
autopct='%1.1f%%') # 绘制拥有车客户性别比例的饼图
fig.add_subplot(2, 2, 3)
data3.plot(kind='pie', title='Customer Children', startangle=60,
autopct='%1.1f%%') # 绘制孩子数的饼图
plt.show()

# 绘制性别和收入的箱线图
gender_data = bank[['income', 'sex']] # 选择性别和收入两列数据
gender_data.boxplot(by='sex', figsize=(6, 6)) # 按性别分组，绘制箱线
图
plt.show()

```

2. 程序说明

- 1) 绘制客户年龄分布的直方图和密度图：
 - 直方图：使用 `plot()` 函数绘制了客户年龄分布的直方图，以展示不同年龄段客户的数量分布。
 - 密度图：使用 `plot()` 函数绘制了客户年龄分布的密度图，以展示年龄分布的平滑曲线。

- 2) 绘制客户年龄和收入的散点图: 使用 `plot()` 函数绘制了客户年龄和收入之间的散点图, 以观察二者之间的关系。
 - 3) 绘制年龄、收入和孩子数的散点矩阵图: 使用 `pd.plotting.scatter_matrix()` 函数绘制了年龄、收入和孩子数之间的散点矩阵图, 以便同时观察它们之间的关系。
 - 4) 按区域分组, 计算收入的均值和标准差, 并绘制柱状图: 程序使用 `groupby()` 函数将客户数据按区域分组, 然后使用 `agg()` 函数计算每个区域的收入均值和标准差。最后, 使用 `plot()` 函数绘制了收入均值的柱状图, 并使用误差线表示标准差, 以进行不同区域客户收入的比较。
 - 5) 绘制性别、有车客户和孩子数的饼图: 程序使用 `groupby()` 函数和 `count()` 函数统计了不同性别、有车客户和孩子数的客户数量, 并使用 `plot()` 函数绘制了对应的饼图。
 - 6) 绘制性别和收入的箱线图: 程序选择了性别和收入两列数据, 并使用 `boxplot()` 函数按性别分组绘制了箱线图, 以展示性别对收入的分布情况。
3. 截图



